

【相关视频】

1. 《大学计算机基础》第二~4周, 国防科技大学 周海芳
2. 计算机科学速成课P3~5, b站



第2'章 计算机中的数

计算机中的存储单位

进位计数制

数制转换

数据在计算机中的表示

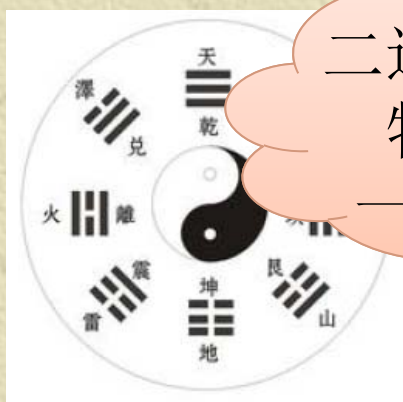
作业

信息在计算机中的表示

✧ **信息**：数值、文字、语音、图形、图像.....

✧ **数据(Data)**：是对信息的一种符号表示，在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称。

- ◆ 在计算机中，所有的信息都将转换成**二进制**数的形式进行存储和运算，即需要将信息进行**二进制的转换**和**编码**。



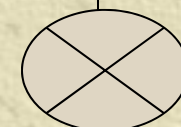
● 二进制与《周易》

二进制便于用某种
物理状态表示
一个抽象的数据

S的两种物理状态：

合 1 开 0

S



二进制的特点

- ✧ 只有两个数码 0 和 1，任何具有两个不同稳定状态的元器件都可用来表示一个二进制位

如：晶体管的截止和导通。

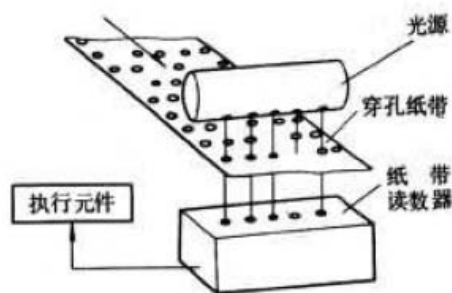
- ✧ 数的存储和传递也可用简单可靠的方法进行

- 理论证明，采用 $R=e=2.71828$ 进制时，存储设备最省。

如：脉冲的有无，电位的高低等。

- ✧ 算术运算规则简单

- ✧ 二进制数的数码 0 和 1，可与逻辑代数中逻辑变量的“假”和“真”对应起来，这样在逻辑运算中可以使用逻辑代数这一数学工具。



计算机中的存储单位

位 (bit, 简称b)

✧ 即一个二进制位，音译为“比特”。

✧ 位是计算机内部数据储存和处理的**最小单位**。

- ◆ 一个二进制位只可以表示0和1两种状态 (2^1)；
- ◆ 两个二进制位可以表示00、01、10、11四种 (2^2) 状态；
- ◆ 三位二进制数可表示八种状态 (2^3)

字节 (byte, 简称B)

✧ 1个字节由8个二进制位构成，音译为“拜特”，
即 $1\text{Byte} = 8\text{bit}$

✧ 字节是计算机中数据存储和处理的基本单位。

◆ 为什么是8位的？据说和Intel 8008的数据总线带宽有关，详见《The C Programming Language》

✧ **存储容量** —— 衡量存储器所能容纳信息量多少的指标。

【单位】字节(Byte, B)

$$1\text{KB} = 1024\text{B} \quad (1024 = 2^{10})$$

$$1\text{GB} = 1024\text{MB} = 1024 \times 1024\text{KB} = 1024^3\text{B}$$

$$1\text{TB} = 1024\text{GB} = 1024^4\text{B}$$

字 (word) *、字长*

✧ 计算机进行数据处理时，作为一个整体进行存取、加工和传送的一串二进制数码，称为一个**计算机字**，简称**字**。

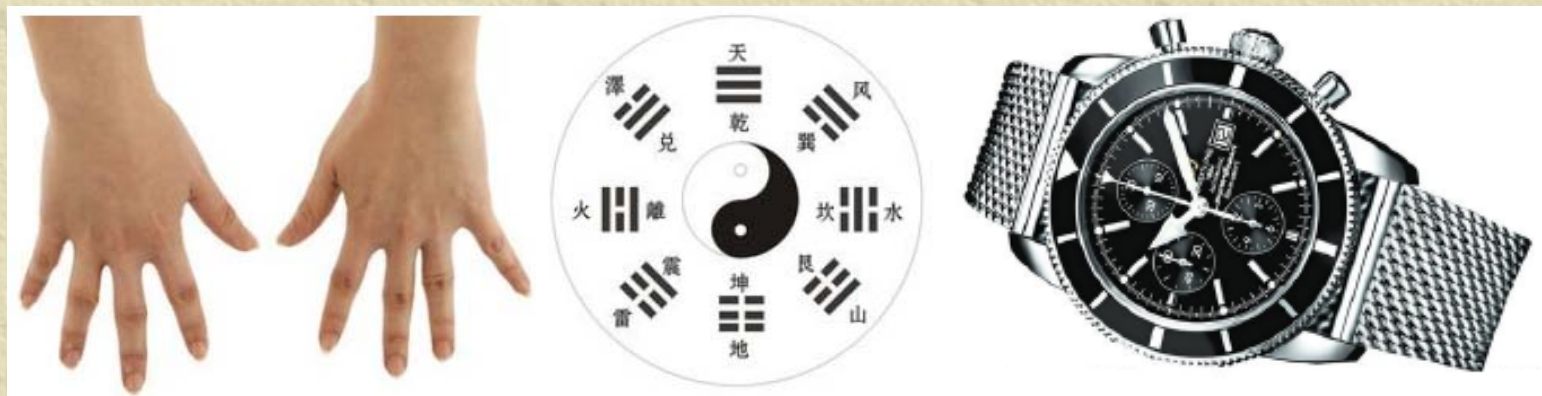
◆ 一个字通常由一个或多个字节构成，即一般是字节的整数倍。

✧ **字长**是指一个计算机字所包含的二进制位数。

◆ 字长直接反映了一台计算机的**计算精度**和**计算性能**，是计算机的一个重要技术指标。

◆ 早期的微机（PC）字长一般是8位和16位；386以及更高的处理器（CPU）大多是32位；目前市面上的计算机的处理器大部分已达到64位。

[返回](#)



进位计数制

- ◆ 十进制数的表示和二进制数的表示
- ◆ 二进制的特点
- ◆ 任意进制数的表示

世界上只有10种人，一种是懂二进制的，另一种是不懂二进制的。

十进制数

✧ 数由 0 ~ 9 这10个不同的符号来表示

【**数码**】用来表示数的符号

【**基数**】表示一位数所需要的不同符号的数目

✧ 计数/运算时由低位向高位进位的规则：
逢十进一

✧ 同一个数码由于它所在的位置不同而有不同的数值。

【**位权/权**】位权的值是基数的若干次幂，相邻位权之比为基数

◆ 某一位的数值 = 该位上的数码 × 该位的位权

$$\begin{aligned}\text{如: } 1978.12 &= 1000 + 900 + 70 + 8 + 1 \times 0.1 + 2 \times 0.01 \\ &= 1 \times 10^3 + 9 \times 10^2 + 7 \times 10^1 + 8 \times 10^0 \\ &\quad + 1 \times 10^{-1} + 2 \times 10^{-2}\end{aligned}$$

十进制数的表示

✧ 基数: 10

✧ 使用数码: 0,1,2,3,4,5,6,7,8,9

✧ 计数规律: 逢十进一

✧ 十进制数N的表示法

◆ 位置记数表示:

$$(N)_{10} = (a_{n-1}a_{n-2}\dots a_1a_0 \cdot a_{-1}a_{-2}\dots a_{-m})_{10}$$

◆ 按权展开式表示:

$$\begin{aligned}(N)_{10} &= a_{n-1} \times 10^{n-1} + a_{n-2} \times 10^{n-2} + \dots + a_1 \times 10^1 \\ &\quad + a_0 \times 10^0 \\ &\quad + a_{-1} \times 10^{-1} + a_{-2} \times 10^{-2} + \dots + a_{-m} \times 10^{-m} \\ &= \sum_{i=-m}^{n-1} a_i \times 10^i\end{aligned}$$

二进制数的表示

✧ 基数：2

✧ 使用数码：0,1

✧ 计数规律：逢二进一

✧ 二进制数N的表示法

◆ 位置记数表示：

$$(N)_2 = (a_{n-1}a_{n-2}\dots a_1a_0 \cdot a_{-1}a_{-2}\dots a_{-m})_2$$

◆ 按权展开式表示：

$$(N)_2 = a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \dots + a_1 \times 2^1 + a_0 \times 2^0 \\ + a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \dots + a_{-m} \times 2^{-m}$$

『例1』 $(111011)_2$

$$= 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 1 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = (59)_{10}$$

『例3』

$$(1101.111)_2$$

$$= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ + 1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

$$= 1 \times 8 + 1 \times 4 + 1 \times 1 + 1 \times 0.5 \\ + 1 \times 0.25 + 1 \times 0.125$$

$$= (13.875)_{10}$$

推广：任意进制数的表示*

✧ 基数： r

✧ 使用数码： $0 \sim r-1$

✧ 计数规律：逢 r 进一

✧ r 进制数 N 的表示法

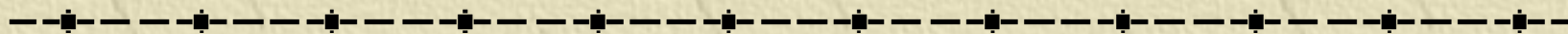
◆ 位置记数表示：

$$(N)_r = (a_{n-1}a_{n-2}\dots a_1a_0 \cdot a_{-1}a_{-2}\dots a_{-m})_r$$

◆ 用按权展开式表示：

$$\begin{aligned}(N)_r &= a_{n-1} \times r^{n-1} + a_{n-2} \times r^{n-2} + \dots + a_1 \times r^1 + a_0 \times r^0 \\ &\quad + a_{-1} \times r^{-1} + a_{-2} \times r^{-2} + \dots + a_{-m} \times r^{-m} \\ &= \sum_{i=-m}^{n-1} a_i \times r^i\end{aligned}$$

常用各种进制数表示法对照表



进制	二进制 Binary	八进制 Octal	十进制 Decimal	十六进制 Hexadecimal
规则	逢二进一	逢八进一	逢十进一	逢十六进一
数码	0,1	0,1,...,7	0,1,...,9	0,1,...,9, A,B,C,D,E,F
基数	$R=2$	$R=8$	$R=10$	$R=16$
位权	2^i	8^i	10^i	16^i
形式表示	B	O或Q	D	H

常用各种进制数的数值对照表

十进制数	二进制数	八进制数	十六进制数
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

常用进制数

✧ 十六进制数的优点

- ◆ 十六进制数简短，便于书写和读数
- ◆ 易转换成二进制数，与计算机本身的结构相适应，所以在微机中应用很普遍，可用来表示机器指令和常数，也可以用来表示各种字符和字母。

✧ 在计算机中书写不同进制数时，常用如下的符号来标识：

C格式控制符

“H”	十六进制数 (Hexadecimal)	%x
“O”	八进制数 (Octal)	%o
“D”	十进制数 (Decimal)	%d
“B”	二进制数 (Binary)	

[返回](#)

数制转换

- ◆ 二进制数和十进制数的转换
- ◆ 八进制数、十六进制数与二进制数的转换

二进制数转换成十进制数

✧ 方法：将二进制数写成按权展开式，并将式中各乘积项的积算出来，然后各项相加，即可得到与该二进制数相对应的十进制数。

『例』

$$\begin{aligned}(11010.101)_2 &= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ &\quad + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= 16 + 8 + 2 + 0.5 + 0.125 \\ &= (26.625)_{10}\end{aligned}$$

十进制数转换成二进制数

✦ 【步骤】 将待转换的数分成整数部分和纯小数部分，并分别加以转换。

整数部分的转换方法——“除2取余”法

✦ 【步骤】把十进制整数除以2，取出余数作为相应二进制数的最低位；把得到的商再除以2，再取余数作为二进制数的次低位；依次类推，继续上述过程，直到商为0，所得余数为最高位。**逆序！**

『例』 $(58)_{10} = (?)_2$

$(58)_{10} = 2(a_{n-1} \times 2^{n-2} + a_{n-2} \times 2^{n-3} + \dots + a_1) + a_0$ ，两边除以2，
得 $(29)_{10} = a_{n-1} \times 2^{n-2} + a_{n-2} \times 2^{n-3} + \dots + a_1$ ，于是取 $a_0 = 0$ ；
又 $(29)_{10} = 2(a_{n-1} \times 2^{n-3} + a_{n-2} \times 2^{n-4} + \dots + a_2) + a_1$ ，两边再除以2，
得 $(14 + 1/2)_{10} = a_{n-1} \times 2^{n-3} + a_{n-2} \times 2^{n-4} + \dots + a_2 + a_1/2$ ，于是取
 $a_1 = 1$

.....

可以采用如下形式进行求解： 因此， $(58)_{10} = (111010)_2$ 。

2|58

2|29.....余数0, a_0

2|14.....余数1, a_1

7.....余数0, a_2

2|7

2|3.....余数1, a_3

2|1.....余数1, a_4

0.....余数1, a_5

返回

纯小数部分的转换方法——“乘2取整”法

✦ 【步骤】把十进制小数乘以2，取其整数作为相应二进制小数的最高位；把乘积的小数部分再乘以2，再取整数作为二进制小数的次高位；依次类推，继续上述过程，直到小数部分为0或达到所要求的精度。

『例』 $(0.625)_{10} = (?)_2$

$(0.625)_{10} = a_{-1}/2 + (a_{-2} \times 2^{-1} + \dots + a_{-m} \times 2^{-m+1})/2$ ，两边乘以2，得 $(1.25)_{10} = a_{-1} + (a_{-2} \times 2^{-1} + \dots + a_{-m} \times 2^{-m+1})$ ，于是取 $a_{-1} = 1$ ；
则 $(0.25)_{10} = a_{-2}/2 + (a_{-3} \times 2^{-1} + \dots + a_{-m} \times 2^{-m+2})/2$ ，再乘以2，得 $(0.5)_{10} = a_{-2} + (a_{-3} \times 2^{-1} + \dots + a_{-m} \times 2^{-m+2})$ ，于是取 $a_{-2} = 0$ ；

.....

可以采用如下形式进行求解：

$$\begin{array}{r} 0.625 \\ \times) \quad 2 \\ \hline 1.250 \dots \text{整数} 1, a_{-1} \\ \times) \quad 2 \\ \hline 0.500 \dots \text{整数} 0, a_{-2} \end{array}$$

$$\begin{array}{r} 0.500 \\ \times) \quad 2 \\ \hline 1.000 \dots \text{整数} 1, a_{-3} \end{array}$$

小数部分为 0

因此， $(0.625)_{10} = (0.101)_2$ 。

纯小数部分的转换方法——“乘2取整”法

『例2』将十进制0.18转换成二进制数，精确到小数点后4位：

0.18

$\times) \underline{2}$

0.36.....整数0, a_{-1}

$\times) \underline{2}$

0.72.....整数0, a_{-2}

$\times) \underline{2}$

1.44.....整数1, a_{-3}

$\times) \underline{2}$

0.88.....整数0, a_{-4}

因此, $(0.18)_{10} \approx (0.0011)_2$ 。

0.88

$\times) \underline{2}$

1.76.....整数1, a_{-5}

达到所要求的精度

[返回](#)

八进制数、十六进制数与二进制数的转换

- ✧ 八进制数的基数是8 ($8=2^3$)，十六进制数的基数是16 ($16=2^4$)。
- ✧ 二进制数、八进制数和十六进制数有2的整指数倍的关系，因而可以直接转换。

常用各种进制数的数值对照表

十进制数	二进制数	八进制数	十六进制数
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

二进制、八进制数相互转化

1位八进制数相当于3位二进制数

■ 二进制→八进制：从小数点向两边每3位为一组，每一组用1位八进制数替换

■ 二进制←八进制：每1位用3位二进制数替换

『例1』 $(10110101.01101)_2 = (?)_8$

小数位数不足3的整数倍的，最右边需补零

$$(10110101.01101)_2 = (\underline{010} \ \underline{110} \ \underline{101} \ \underline{011} \ \underline{010})_2$$

$$= (2 \ 6 \ 5 \ . \ 3 \ 2)_8$$

二进制、十六进制数相互转化

1位十六进制数相当于4位二进制数

- 二进制→十六进制：从小数点向两边每4位为一组，每一组用1位十六进制数替换
- 二进制←十六进制：每1位用4位二进制数替换

『例2』 $(10110101.01101)_2 = (?)_{16}$

小数位数不足4的整数倍的，最右边需补零

$$(10110101.01101)_2 = (\underline{1011} \ \underline{0101} . \underline{0110} \ \underline{1000})_2$$

$$= (\begin{matrix} \updownarrow & \updownarrow & \updownarrow & \updownarrow & \updownarrow \\ \text{B} & 5 & . & 6 & 8 \end{matrix})_{16}$$

八进制、十六进制数相互转化

以二进制为中介

『例3』

$$\begin{aligned} & (12F)_{16} \\ &= (0001\ 0010\ 1111)_2 \\ &= (000\ 100\ 101\ 111)_2 \\ &= (157)_8 \end{aligned}$$

$$\begin{aligned} & (AF.16C)_{16} \\ &= (1010\ 1111.0001\ 0110\ 11)_2 \\ &= (010\ 101\ 111.000\ 101\ 101\ 10\mathbf{0})_2 \\ &= (257.0554)_8 \end{aligned}$$

返回

数据在计算机中的表示

实数的表示
字符的表示

Q&A



✧ 什么是数据类型？

- ◆ 针对不同类型的数据，C语言采用不同的存储和处理方式。
- ◆ C语言中的数据可分为“数值”和“字符”两大类型。

✧ 为什么要把数值类型进一步分为“整型”和“实型”两种？

- ◆ 两者的存储空间大小、编码方案和运算方法不同

✧ 为什么计算机中能表示的数值范围都是有限的？

- ◆ 因为存储空间大小（二进制位数）有限。
- ◆ 正因为这一点，编程中应注意“数值精度”、“溢出”等问题！

1000

'A'

1001

3255

1002

1003

1004

113.3

1005

1006

⋮

实数在计算机中的表示

- ✧ 真值与机器数
- ✧ 有符号数、无符号数
- ✧ 原码、反码*和补码
- ✧ 机器数的加、减运算*
- ✧ 机器数的三种代码表示的比较
- ✧ 数的定点表示和浮点表示*
 - ◆ 数的定点表示*
 - ◆ 数的浮点表示*

• 打*号部分不做要求，可作为扩展阅读！

真值与机器数

✧ 带符号数字的组成部分：

<数的符号> <数的数值>

如：19、+12、-15

✧ 【真值】直接用正号“+”和负号“-”来表示符号的二进制数，如+1101、-1001，不能直接用于数字计算机。

✧ 【机器数】将真值中的符号数值化（即，用0表示正，用1表示负）后的二进制数，可以为计算机所用。

◆ 一般地，n个二进制位的机器数，左起第一位表示符号，剩余n-1位表示数值。

有符号数、无符号数

✧ 在计算机中，可以区分正负的**数值类型**，称为**有符类型**，无正负的数值类型（只有正值），称为**无符类型**。

◆ 如：年龄、体重、库存量

✧ C语言中，数值类型分为**整型**和**实型**，其中整型又分为无符类型或有符类型，而实型则只有有符类型。

✧ C语言中，字符类型也分为有符和无符类型。

有符号数、无符号数 (2)

✧ 在计算机中，当一个数值是有符号类型时，其最高数位称为“**符号位**”。

◆ 符号位为1时，表示该数为负值，为0时表示为正值。

✧ 当一个数值是无符号类型时，那么其最高位的1或0，和其它位一样，用来表示该数的大小。

3个比特能表示的有符号数：

$$(000)_2 = +0, (100)_2 = -0$$

$$(001)_2 = +1, (101)_2 = -1$$

$$(010)_2 = +2, (110)_2 = -2$$

$$(011)_2 = +3, (111)_2 = -3$$

3个比特能表示的无符号数：

$$(000)_2 = +0, (100)_2 = +4$$

$$(001)_2 = +1, (101)_2 = +5$$

$$(010)_2 = +2, (110)_2 = +6$$

$$(011)_2 = +3, (111)_2 = +7$$

无符号数和有符号数的范围区别

✧ 以一个字节的数值（即8个二进制位）为例

◆ 最大值

无符号数: 1111 1111 值: 255

$$1 * 2^7 + 1 * 2^6 + 1 * 2^5 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$$

有符号数: 0111 1111 值: 127

$$1 * 2^6 + 1 * 2^5 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$$

- 原因是有符号数中的最高位被挪去表示符号了，少掉最高位，其最大值一下子减半。

◆ 最小值

无符号数: 0000 0000 值: 0

有符号数: 1000 0000 值: -128

◆ 取值范围

无符号数: 0 ----- 255

有符号数: -128 ----- 0 ----- 127

- 二者能表达的不同数值的个数都一样是256个！

原码、反码*和补码——背景

- ✧ 机器数在计算机中要进行加、减、乘、除等运算。
- ✧ 实际上，乘法是做移位加法，除法是做移位减法。
- ✧ 因此，在机器中只需要做加、减两种运算。
- ✧ 如果直接实现减法，必须使用较复杂的逻辑电路来实现，运算时间较长。
- ✧ 于是，人们提出了多种机器数的表示形式，试图用加法运算来实现减法运算。
 - ◆ 几乎所有现代计算机的数值表示都采用补码！

原码表示法

✧ 原码表示也称为“**符号—数值表示**”。

✧ 形成规则：

- ◆ **最高位表示符号**（正数符号位为0，负数符号位为1）
- ◆ 其余位表示数值部分，为**数值绝对值**的二进制表示。

✧ 整数（以8位为例）

$$x = 1100110, \quad [x]_{\text{原}} = 01100110$$

$$x = -1100110, \quad [x]_{\text{原}} = 11100110$$

$$\text{◆ } [+0]_{\text{原}} = 00000000, \quad [-0]_{\text{原}} = 10000000$$

✧ 纯小数（以8位为例）

$$x = 0.1100110, \quad [x]_{\text{原}} = 0.1100110$$

$$x = -0.1100110, \quad [x]_{\text{原}} = 1.1100110$$

$$\text{◆ } [+0]_{\text{原}} = 0.0000000, \quad [-0]_{\text{原}} = 1.0000000$$

反码表示法*

✧ 反码表示也称为“对1的补数”。

✧ 形成规则：

- ◆ 正数的反码表示完全等同于其原码表示；
- ◆ 对于负数，最高位为1（符号位），其余位（即数值部分）为相应原码表示数值部分按位求反。

✧ 整数（以8位为例）

$$x = 1100110, \quad [x]_{\text{反}} = 01100110$$

$$x = -1100110, \quad [x]_{\text{反}} = 10011001$$

$$\text{◆ } [+0]_{\text{反}} = 00000000, \quad [-0]_{\text{反}} = 11111111$$

✧ 纯小数（以8位为例）

$$x = 0.1100110, \quad [x]_{\text{反}} = 0.1100110$$

$$x = -0.1100110, \quad [x]_{\text{反}} = 1.0011001$$

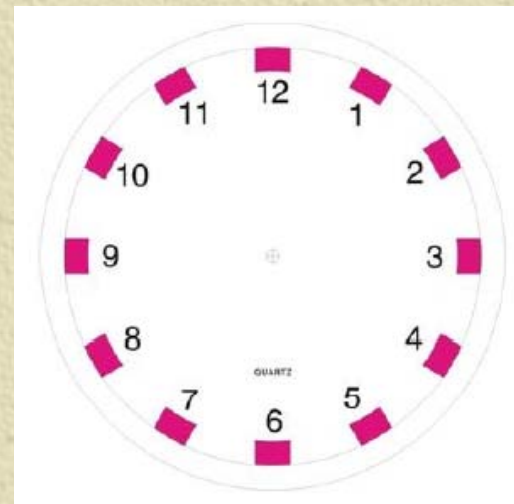
$$\text{◆ } [+0]_{\text{反}} = 0.0000000, \quad [-0]_{\text{反}} = 1.1111111$$

补码的由来：模和补数

✧ **模**：一个计量系统的计数范围

◆ 比如时钟，模=12

- 计数总是不会超过12
- 12点过后，就是1点



✧ **补数**：两个小于模的正数，和为模则互为补数

◆ 当模=12，1和11，2和10，...，6和6 互为补数

补码的由来：带模的计算

✧ 带模的计量系统

- ◆ 运算结果超出计数范围则表示为模的余数
 - 现假设模为12：
 - ◆ $6+7=13 \rightarrow (\text{对12求余数}) 1$
 - ◆ $8+10=18 \rightarrow (\text{对12求余数}) 6$
- ◆ 可以化减法为加法：
 - 减一个数等价于加上它的补数（设模为12）
 - ◆ $9-4$ 相当于 $9+8=17 \rightarrow (\text{对12求余数}) 5$
 - ◆ $6-1$ 相当于 $6+11=17 \rightarrow (\text{对12求余数}) 5$
- ◆ 好处：做加减法不需要判断符号位！

补码表示法

★ 编码规则:

- ◆ 正数的补码表示完全等同于其原码表示;
- ◆ 对于负数, 最高位为1 (符号位), 其余位 (即绝对值) **按位取反**, 得到的结果最低位**再加1**。

★ 整数 (以8位为例)

$$x = 1100110,$$

$$[x]_{\text{补}} = 01100110$$

$$x = -1100110,$$

$$[x]_{\text{补}} = 10011010$$

- ◆ $[+0]_{\text{补}} = [-0]_{\text{补}} = 00000000$

★ 纯小数 (以8位为例)

$$x = 0.1100110,$$

$$[x]_{\text{补}} = 0.1100110$$

$$x = -0.1100110,$$

$$[x]_{\text{补}} = 1.0011010$$

- ◆ $[+0]_{\text{补}} = [-0]_{\text{补}} = 0.0000000$

绝对值相等的正值和负值的转换关系:

- 1) 从右向左复制, 直到复制了一个1为止;
- 2) 对剩余的部分按位取反;

机器数的不同编码表示的比较

✧ 原码表示：简单方便，但减法运算必须真正实施，不能用加法运算代替，于是增加了实现所需的逻辑电路的复杂度。此外，还有一个缺点即零的编码有两种。

◆ 0的原码（以 8 位整数为例）： 10000000 和 00000000

✧ 反码表示*：运算实现较简单，减法运算可用加法运算代替。但是运算中若出现进位则需要两次算术相加。另外，和原码一样，它的零也有两种编码。

✧ 补码表示：运算实现较简单，减法运算可用加法运算代替。运算中出现进位处理简单。而且零的编码只有一种。

◆ 0的补码（以 8 位整数为例）： 00000000

对于 **N** 位的二进制数

● 原码表示范围： $-(2^{N-1}-1) \sim (2^{N-1}-1)$

● 补码表示范围： $-2^{N-1} \sim (2^{N-1}-1)$

-2^{N-1} 是孤独的， $[-2^{N-1}]_{\text{补}} = (100 \dots 0)_2$

[返回](#)

机器数的三种代码表示的比较 (2) *

✧ 表示范围

◆ 原码、反码*:

8位整数: $1-2^7-1 \sim 2^7-1$, 即 $-127 \sim 127$

原码: $11111111 \rightarrow 10000000$ 、 $00000000 \rightarrow 01111111$

8位小数: $2^{-7}-1 \sim 1-2^{-7}$, 即 $-127/128 \sim 127/128$

原码: $1.1111111 \rightarrow 1.0000000$ 、 $0.0000000 \rightarrow 0.1111111$

◆ 补码:

8位整数: $-2^7-1 \sim 2^7-1$, 即 $-128 \sim 127$

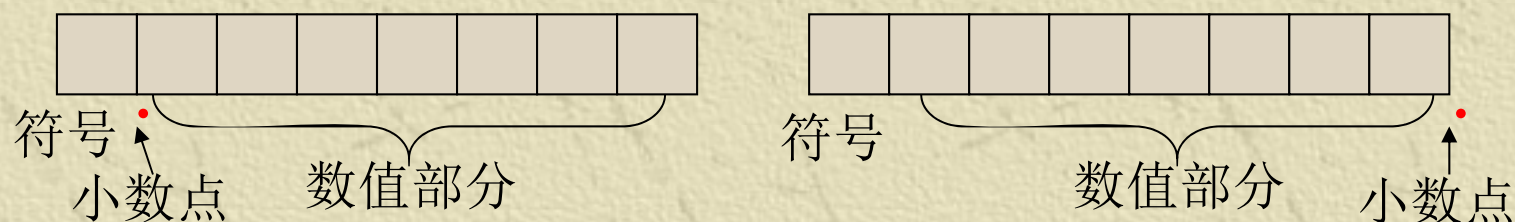
$10000000 \rightarrow 11111111 \rightarrow 00000000 \rightarrow 01111111$

8位小数: $2^{-7}-1 \sim 1-2^{-7}$, 即 $-127/128 \sim 127/128$?

$1.0000000? \rightarrow 1.1111111 \rightarrow 0.0000000 \rightarrow 0.1111111$

数的定点表示*

- ✧ **定点表示**：小数点在数中的位置固定不变。
- ✧ **定点机**：使用定点数的计算机。
- ✧ 通常把小数点固定在数的符号位之后或固定在最低位之后。



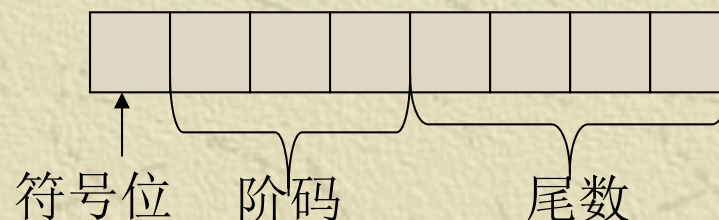
- ✧ n 位(不算符号位)定点小数 N 的值域为：
$$2^{-n} \leq |N| \leq 1 - 2^{-n}$$
- ✧ n 位(不算符号位)定点整数 N 的值域为：
$$0 \leq |N| \leq 2^n - 1$$
- ✧ 定点机中，运算数和运算结果都不能超出数域范围，否则出现“溢出”（**上溢**或**下溢**）。

数的浮点表示*

✦ 浮点表示：小数点在数中的位置不固定。

✦ 一般表示形式： $N=2^J \times S$

其中， S 为 N 的尾数， J 为阶码，2为 J 的基数



✦ 浮点数中，阶码部分反映了小数点浮动的位置，即数值范围；尾数部分则体现数的有效数位。

✦ 为提高运算精度，避免有效数字丢失，浮点数字一般表示成规格化数（使 $1/2 \leq |S| < 1$ ），例如： $2^{101} \times 0.0101 \rightarrow 2^{100} \times 0.1010$

数的浮点表示 (2) *

『例』 设16位浮点表示法，用5位补码表示阶码，11位原码表示尾数，则

$$N_2 = 1010.110101 = 0.\underline{1010110101} * 2^{0100}$$

表示为: 0 00100 10101101001

$$N_3 = -0.0011010011 = -0.\underline{1101001100} * 2^{-0010}$$

表示为: 1 11110 1101001100

『思考』 此种浮点表示方案的表示范围如何？

【机器零】 尾数为0（不论阶码为何值），或者，阶码等于最小值（不论尾数为何值）

【舍位错误】 由于尾数长度不足致使存储的数值部分地丢失。

- ◆ 在数值相加时，他们相加地次序可能是重要的。因为若一个很大的数加一个非常小的数，那么小的数可能会被舍去。

字符在计算机中的表示

- ✧ 相关背景
- ✧ ASCII码
- ✧ Unicode*
- ✧ BCD码*
- ✧ 汉字编码*

字符在计算机中的表示——背景

- ✧ 字母及各种字符必须按特定的规则变为二进制编码才能进入计算机。
 - ◆ 计算机对字符的处理实际上是对字符的内部编码进行处理。
- ✧ **字符编码**实际上就是为每一个字符确定一个对应的整数值（以及它对应的二进制编码）。
 - ◆ 不同的字符对应不同的整数值，反之亦然。
 - ◆ 为了在计算机中使用的方便，字符的编码都是从0开始，连续排列的。
 - ◆ 由于字符（包括拉丁字母等）与整数值之间没有什么必然的联系，某一个字符究竟对应哪个整数完全可以人为地规定。
- ✧ 为了信息交换中的统一性，人们建立了一些**字符编码标准**，常用的有ASCII码、Unicode等等。

ASCII码

✧ **ASCII码**（American Standard Code for Information Interchange，美国标准信息交换码）是国际上广泛使用的字符代码。（详见P377附录B）

- ◆ 字符宽度为8位，最高位为0，余下的7位可以给出128个编码，用来表示128种不同的字符。其中的95个编码，对应键盘上能敲入并且可以显示和打印的95个字符。
- ◆ 常用的ASCII码：

空格—32	0~9—48~57
A~Z—65~90	a~z—97~112
- ◆ 标准ASCII码只使用了低7位，当最高位为1时，又可引出128~255共128个编码，这些编码怎样定义并未标准化，用来在特定的计算机上定义其它字符，如**汉字编码**。

Unicode*

- ✧ ASCII码的局限性：字符采用8位存储，仅能表示最多256个不同的字符，表示中文、日文等远东语言有困难。
- ✧ 统一代码联盟提出了字符表示的新标准——**统一代码（Unicode）**
 - ◆ 字符的宽度是字节的整数倍，为每种语言中的每个字符设定了统一并且唯一的二进制编码。

BCD码*

BCD码——用4位二进制数表示1位十进制数的编码方式，具体有：8421码、2421码、余三码、格雷码等等。

例：十进制： 8 3 1

 ↑ ↑ ↑

 ↓ ↓ ↓

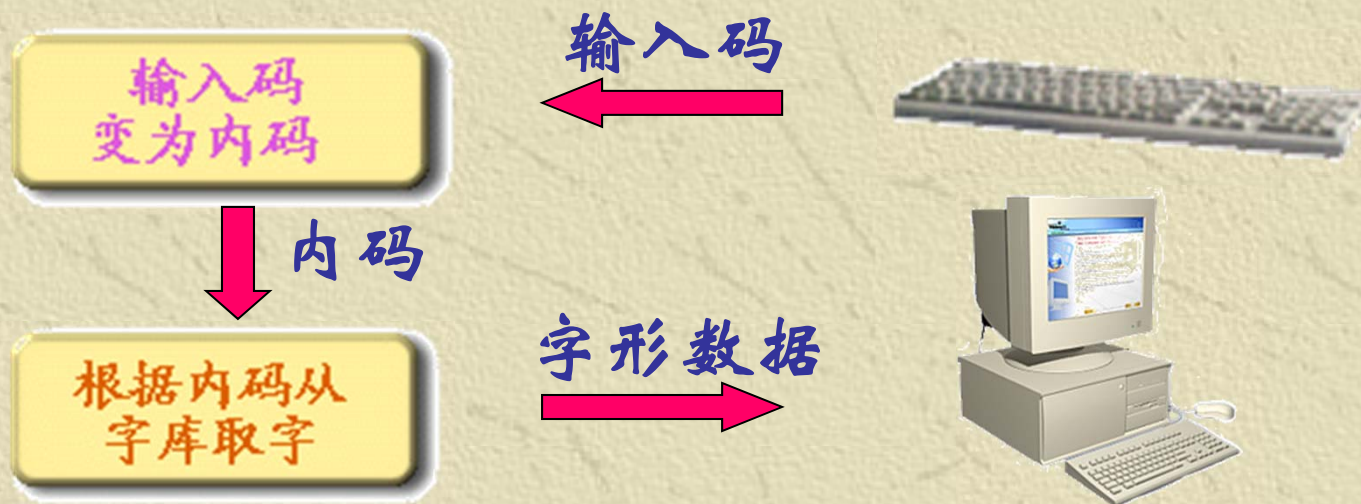
8421BCD码： 1000 0011 0001

汉字编码*

汉字输入码：区位码、拼音码、五笔字型码

汉字内码：计算机内部进行存储、传输和加工时所用的统一机内代码，如：根据GB2312-80图形字符分区表制定的编码。

汉字字形码：汉字字形点阵中一个点用一位二进制码表示。



汉字点阵*

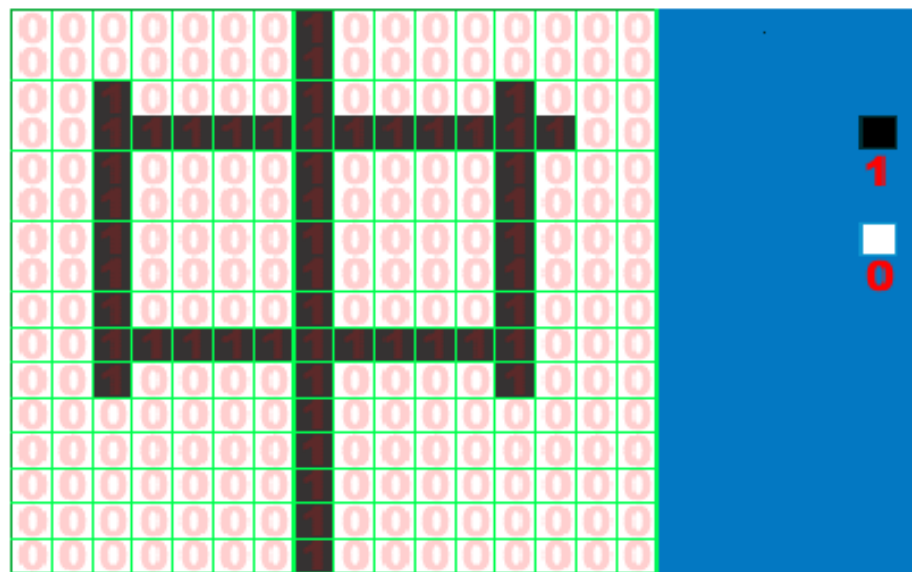
“中”字的内
码

D6D0

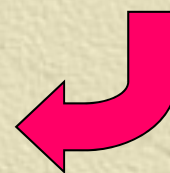


0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	3	F	F
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	3	F	8
0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0

字库中存储的字型数据



屏幕
显示
方式



[返回](#)

作业

1. 将下列十进制数，转换成二进制数，再转换成八和十六进制。

(1) 67

(2) 253

(3) 1024

(4) 218.875

(5) 0.0625

2. 将下列不同进制数写成按权展开形式，并分别转换成十进制数。

(1) $(10110.01101)_2$

(2) $(5701.36)_8$

(3) $(8D4A.1FC)_{16}$

作业 (2)

3. 写出下列各数的原码和补码。(以1Byte为例)

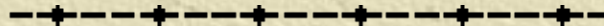
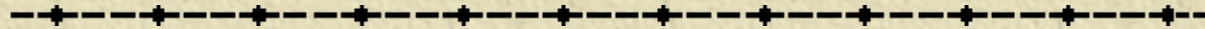
(1) 0.1011

(2) -10110

(3) -0.1010



The End



二进制数的算术运算*

运算法则:

$0+0=0$	$0+1=1$	$1+0=1$	$1+1=10$ (进位)
$0-0=0$	$0-1=1$ (借位)	$1-0=1$	$1-1=0$
$0\times 0=0$	$0\times 1=0$	$1\times 0=0$	$1\times 1=1$
$0\div 0=0$	$0\div 1=0$	$1\div 0=0$ (无意义)	$1\div 1=1$

『例1』 $1101+1011$

	被加数	1	1	0	1	
+)	加数	1	0	1	1	
	进位	1	1	1	1	
	和	1	1	0	0	0

『例2』 1101×1011

				1	1	0	1
×)				1	0	1	1
				1	1	0	1
			1	1	0	1	
		0	0	0	0		
	1	1	0	1			
	1	0	0	0	1	1	1

首要步骤: 按数位对齐(右对齐)

二进制数的算术运算 (2) *

算术性质：

- 1) 奇偶性质：最低位为0---偶数；为1---奇数
- 2) 移位性质：小数点右移---乘2；左移---除2
 - 将加法运算和部分积右移的方法结合起来即可实现乘法运算。

返回

逻辑变量与基本逻辑运算*

✧ 逻辑变量的取值只有0(假)或1(真)两种。

✧ 三种基本逻辑运算

◆ 或：逻辑问题中，如果决定某一事件发生的多个条件中，只要有一个或一个以上成立，事件便可发生，则这种因果关系称之为“或”逻辑。

◆ 与：逻辑问题中，如果决定某一个事件发生的多个条件必须同时具备，事件才能发生，则这种因果关系称之为“与”逻辑。

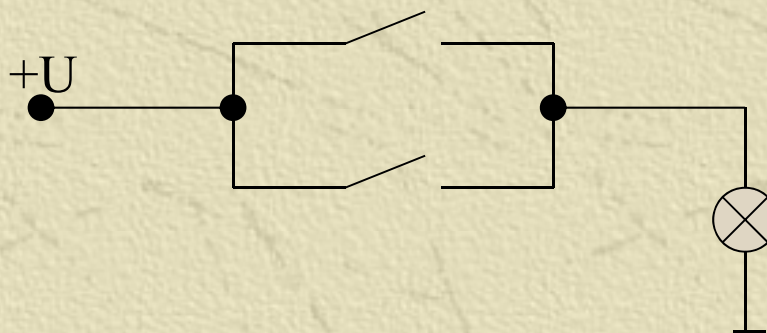
◆ 非：逻辑问题中，如果某一时间的发生取决于条件的否定，即事件与事件发生的条件之间构成矛盾，则这种因果关系称为“非”逻辑。

✧ 其它逻辑运算，如：异或、同或

返回

或运算*

✧ 用电路来理解



✧ 运算符： $\parallel$ （数学上用 $\vee$ 、OR）

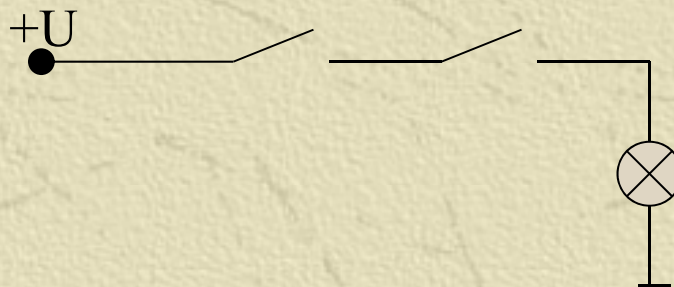
✧ 运算规则： $0 \parallel 0 = 0$ $1 \parallel 0 = 1$

$0 \parallel 1 = 1$ $1 \parallel 1 = 1$

[返回](#)

与运算*

✧ 用电路来理解



✧ 运算符：&&（数学上用 $\wedge$ 、AND）

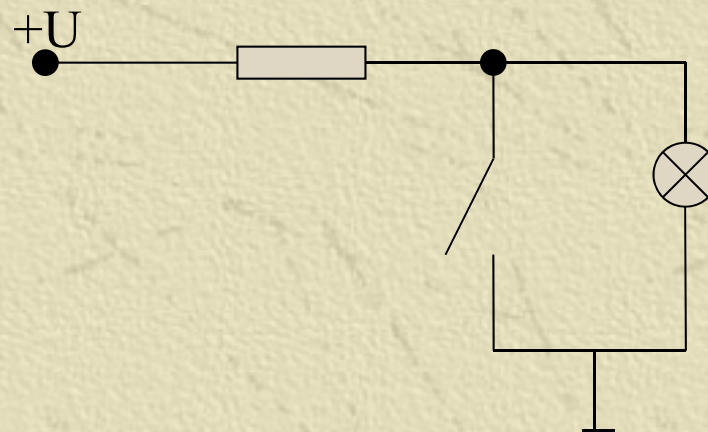
✧ 运算规则：

$0 \&\& 0 = 0$	$1 \&\& 0 = 0$
$0 \&\& 1 = 0$	$1 \&\& 1 = 1$

[返回](#)

非运算*

✧ 用电路来理解



✧ 运算符：！ （数学上用 $\neg$ 、NOT）

✧ 运算规则：！ 0=1 ！ 1=0

返回

异或、同或运算*

✧ 异或运算

◆ 运算符: $\oplus$ 、.XOR

◆ 运算规则: $0 \oplus 0=0$ $1 \oplus 0=1$ 两者相异则为1!
 $0 \oplus 1=1$ $1 \oplus 1=0$

◆ $A \oplus B = (A \wedge \bar{B}) \vee (\bar{A} \wedge B)$

✧ 同或运算

◆ 运算符: $\odot$

◆ 运算规则: $0 \odot 0=1$ $1 \odot 0=0$ 两者相同则为1!
 $0 \odot 1=0$ $1 \odot 1=1$

◆ $A \odot B = (A \wedge B) \vee (\bar{A} \wedge \bar{B})$

返回